

1. Mi az a Node.js? Terminál és CLI alapok

Miért jött létre a Node.js?

Sokáig a JavaScript egy olyan nyelv volt, ami be volt zárva a böngészőbe. Csak arra tudtuk használni, hogy a weboldalakat interaktívvá tegyük (pl. gombnyomásra lenyíló menü). 2009-ben Ryan Dahl fogta a Google Chrome böngésző JavaScript motorját (a V8 motort), ami a JS kódot gépi kóddá fordítja, és kiemelte a böngészőből. Hozzáírt egy C++ alapú keretrendszert, hogy a JavaScript képes legyen fájlokat olvasni, hálózathoz csatlakozni és adatbázisokkal kommunikálni. Így született meg a Node.js, ami lehetővé tette, hogy a szerver (backend) oldalt is JavaScriptben írjuk meg.

Hogyan működik? A pincér hasonlat (Non-blocking I/O)

Képzeljünk el egy éttermet egyetlen pincérrel (ez a Node.js egyetlen szála). Ha a hagyományos modellt követné (blokkoló), a pincér felvenné a rendelést az 1. asztalnál, bevinné a konyhába, és addig állna a konyhaajtóban várva az ételre, amíg az el nem készül. Közben a többi asztal hiába várna rá. A Node.js aszinkron (nem blokkoló) módon működik: a pincér felveszi a rendelést, leadja a konyhán, és azonnal megy a 2. asztalhoz. Amikor a szakács (háttérfolyamat) elkészült az étellel, szól a pincérnek (Event Loop), aki kiviszi azt. Így egyetlen szállal is hatalmas forgalmat tudunk kiszolgálni, gyorsan és hatékonyan.

Terminál (Parancssor) Alapok

Mivel a backend rendszereknek általában nincs vizuális felülete (nincs gomb, amire rákattintunk), mindent parancsokkal irányítunk a terminálban.

- **pwd / cd:** Hol vagyok épp a fájlrendszerben? (Linux/Mac: Print Working Directory, Windows: Current Directory)
- **mkdir <mappanév>:** Könyvtár (mappa) létrehozása (Make Directory).
- **cd <útvonal>:** Navigálás a mappák között. Például 'cd asztal' belép, 'cd ..' visszalép egy szintet.
- **npm:** A Node Package Manager (Csomagkezelő). Ezen keresztül tudunk letölteni mások által megírt hasznos kódokat, modulokat az internetről (pl. jelszó titkosító, képfeldolgozó).

2. Node.js futtatása (node app.js), script végrehajtás

Mi történik a háttérben?

A JavaScript egy úgynevezett 'interpretált' nyelv. Ez azt jelenti, hogy nem kell előre lefordítanunk (fordítani/compile-olni) egy futtatható .exe fájlra, mint például a C#-ot vagy a C++-t. Amikor beírjuk a terminálba, hogy 'node app.js', a Node.js elkezd olvasni a fájlt, sorról sorra. A kódokat azonnal lefordítja gépi utasításokká a V8 motor segítségével, és rögtön végre is hajtja. Amikor a fájl legvégére ér, és már nincsenek háttérben várakozó feladatok, a program befejezi a futását, és visszakapjuk a parancssort.

Futtatható példa: app.js

```
// --- app.js ---
console.log("Szia! Ez az első Node.js programom.");

let a = 5;
let b = 10;
let osszeg = a + b;

console.log("A két szám összege:", osszeg);
console.log("A program most befejeződik.");

// Futtatás a terminálban:
// node app.js
```

3. Változók és adattípusok a szerveroldalon

A const rejtett tulajdonsága: Miért nem 100%-ig állandó?

A modern JavaScriptben szinte mindenre a 'const' (állandó/konstans) kulcsszót használjuk, mert ez teszi a kódunkat kiszámíthatóvá. A fejlesztők pontosan tudják, hogy a változó nem fog új értéket kapni. Azonban van egy kis csavar! A 'const' csak az úgynevezett 'újraértékelést' tiltja meg. Képzeljük el úgy, hogy a 'const' egy felcímkézett doboz. A doboz címkéjét (nevét) és magát a dobozt nem cserélhetjük le egy másik dobozra. De ha a dobozba egy objektumot vagy tömböt teszünk, a doboz kinyitása után az abban lévő tartalom (pl. egy diák jegye) módosítható!

Futtatható példa: variables.js

```
// --- variables.js ---
// 1. PRIMITÍV TÍPUSOK (Számok, szövegek)
const nev = "Gábor";
let életkor = 17;

életkor = 18; // Ez működik, mert 'let'-tel hoztuk létre
// nev = "Péter"; // EZ HIBA LENNE! A 'const' nem írható felül!

console.log("Név:", nev, "Kor:", életkor);

// 2. OBJEKTUMOK és a CONST viselkedése
const diak = {
  nev: "Anna",
  osztaly: "10.B"
};

// Ez HIBA lenne (a 'dobozt' magát nem cserélhetjük ki):
```

```
// diak = { nev: "Peti" };

// DE a doboz tartalmának módosítása ENGEDÉLYEZETT!
diak.osztaly = "11.B";
diak.jegy = 5;

console.log("Módosított diák adatai:", diak);

// Futtatás: node variables.js
```

4. Konzol használata, hibakeresés (console.log, console.error)

A konzol a backend fejlesztő 'szeme'

Amikor weboldalt készítünk (frontend), ha egy gomb rosszul jelenik meg, azt azonnal látjuk. A backend viszont a háttérben fut (egy adatközpont szerverén, monitor nélkül). Ha egy kérés beérkezik a szerverünkhöz, és leáll valami miatt a program, honnan fogjuk tudni, hogy mi történt? Itt jön képbe a konzol (console). A console parancsok a standard kimenetre (Standard Output - stdout) és a hibakimenetre (Standard Error - stderr) írnak. A valós életben ezeket a logokat egy szöveges fájlba mentjük, amit a fejlesztő később vissza tud olvasni, akár egy repülőgép feketedobozát.

Futtatható példa: logging.js

```
// --- logging.js ---
// 1. Általános logolás (információk kiírása)
// Olyan, mintha a szerver naplót vezetne a tevékenységéről
console.log("Szerver elindult. Minden rendben.");

// 2. Hibák logolása
// Sok terminál ezt piros színnel emeli ki. Fontos a hibakeresésnél!
console.error("Hiba történt: Nem található az adatbázis!");

// 3. Figyelmeztetés (Gyakran sárga színű)
console.warn("Figyelem: Kevés a memória!");

// 4. Táblázatos megjelenítés
// Ha sok adatunk van (pl. egy lista az adatbázisból), ez átláthatóvá teszi.
const diakok = [
  { nev: "Anna", jegy: 5 },
  { nev: "Peti", jegy: 3 },
  { nev: "Kati", jegy: 4 }
];
```

```
console.log("\nDiákok listája:");
console.table(diakok);

// Futtatás: node logging.js
```

5. Logikai és aritmetikai műveletek

A 'Truthy' és 'Falsy' értékek fogalma

Egy 'if' (ha) feltételben általában igaz (true) vagy hamis (false) értékeket vizsgálunk (pl. `if(10 > 5)`). De mi történik, ha egy 'if' feltételbe egy szöveget vagy egy számot írunk be? A JavaScript rendelkezik egy szabályrendszerrel, ami alapján minden létező adatot át tud alakítani logikai értéké. Ezt hívjuk 'Truthy' (igaznak számító) és 'Falsy' (hamisnak számító) értékeknek. Ezt a tudást arra használjuk, hogy nagyon röviden tudjunk ellenőrizni dolgokat, például azt, hogy egy felhasználó kitöltött-e egy beviteli mezőt, vagy üresen hagyta (üres szöveg = hamis).

- **Falsy értékek listája:** false, 0 (nulla), "" (üres string), null, undefined. Ha ezeket vizsgáljuk, a program hamisként fogja őket kezelni.
- **Truthy értékek:** Minden, ami nem szerepel a fenti listában! (Pl. a "Szia" szöveg, vagy a 100-as szám).

Futtatható példa: logic.js

```
// --- logic.js ---
const isBejelentkezve = true;
const username = ""; // Üres szöveg -> Falsy (hamisnak számít)

// 1. Rövidzárlat ÉS (&&) operátorral
// Ha az operátor bal oldala IGAZ, csak akkor hajtja végre a jobb oldalt.
// Ez egy nagyon profi, rövidített if() feltétel!
isBejelentkezve && console.log("Üdvözlöm a rendszerben!");

// 2. Alapértelmezett érték VAGY (||) operátorral
// Ha a bal oldal hamis (pl. üres szöveg), akkor a jobb oldalt adja vissza.
// Így tudunk könnyen alapértelmezett értékeket beállítani.
const felhasznaloNev = username || "Ismeretlen Felhasználó";
console.log("Bejelentkezett név:", felhasznaloNev);

// 3. Nullish Coalescing (??) operátor
// Hasonló a || operátorhoz, de csak a 'null' vagy 'undefined' esetén lép életbe.
const pontszam = 0;
// A || operátor a 0-t is hamisnak venné, és kicserélné. A ?? viszont megtartja a 0-t!
const veglegesPont = pontszam ?? 10;
console.log("Pontszám:", veglegesPont); // 0 lesz az eredmény
```

```
// Futtatás: node logic.js
```

6. Feltételvizsgálatok a szerverlogikában

A 'Guard Clause' (Őrfeltétel) koncepció

Sok kezdő programozó úgy írja az 'if' (ha) feltételeket, hogy a pozitív ágot teszi előre, és ha valami nem jó, egy beágyazott 'else' (különben) ágba rakja a hibaüzenetet. Ha 3-4 feltételt kell vizsgálni (pl. be van-e jelentkezve, van-e elég pénze, raktáron van-e a termék), akkor a kód egyre beljebb csúszik (ezt hívják 'Nyíl' vagy 'Spagetti' kódnak). A profi backend fejlesztők 'Őrfeltételeket' (Guard Clauses) használnak. Képzeljük el az őrfeltételt úgy, mint a biztonsági őrt a klub bejáratánál. Aki nem felel meg a feltételnek, azt az őr azonnal kiküldi (return), és nem is mehet tovább. Akik viszont áthaladnak az összes őron, azok bejutnak a 'boldog ágra' (Happy Path), ahol már nincsenek felesleges beágyazások.

Futtatható példa: conditions.js

```
// --- conditions.js ---
// Rossz gyakorlat: Beágyazott if-else (Kezdő, Spagetti stílus)
function belepesRossz(kor) {
  if (kor !== null) {
    if (kor >= 18) {
      console.log("Rossz: Sikeres belépés!");
    } else {
      console.log("Rossz: Túl fiatal vagy!");
    }
  } else {
    console.log("Rossz: Nem adtál meg kort!");
  }
}

// Jó gyakorlat: Guard Clauses (Őrfeltételek) - Profi, Tiszta kód
function belepesJo(kor) {
  // 1. biztonsági őr: Megadott egyáltalán kort?
  // Ha nem, azonnal kilépünk a return kulcsszóval.
  if (kor == null) return console.log("Hiba: Nem adtál meg kort!");

  // 2. biztonsági őr: Elég idős?
  if (kor < 18) return console.log("Hiba: Túl fiatal vagy a belépéshez!");

  // Happy Path (Boldog ág) - Aki idáig eljut a kódban, az tuti érvényes!
  console.log("Szuper: Sikeres belépés a rendszerbe!");
}
```

```
belepesJo(20); // Ez a "Happy path"-ra fut
belepesJo(15); // Ez fennakad a 2. órön

// Futtatás: node conditions.js
```

7. Moduláris kódolás alapjai Node.js-ben

Miért daraboljuk fel a kódot?

Képzeljük el, hogy egy autót építünk. Nem egyetlen hatalmas, egybefüggő fémdarabból öntjük ki az egészet, hanem alkatrészekből (motor, kerekek, ajtók) szereljük össze. Ha a motor elromlik, elég csak azt kicserélni, nem kell eldobni az egész autót. A programozásban ez a modularitás. Hogy ne legyen egyetlen hatalmas, 10 000 soros kóduk (amit lehetetlen átlátni), a kódot külön fájlokba (modulokba) szervezzük. Az egyik fájl felel a matek műveletekért, a másik az adatbázisért. A Node.js hagyományosan a 'CommonJS' modulrendszert használja, amivel a 'module.exports' paranccsal publikussá tesszük egy fájl tartalmát, a 'require()' paranccsal pedig áthúzzuk (beimportáljuk) egy másikba.

Példa (CommonJS - Hagományos Node.js mód)

Két fájlt kell létrehozni ugyanabban a mappában:

```
// --- matek.js (Az alkatrész / modul) ---
const osszeadas = (a, b) => a + b;
const kivonas = (a, b) => a - b;

// Kifelé láthatóvá tesszük a függvényeinket (exportáljuk)
module.exports = {
  osszeadas,
  kivonas
};

// --- index.js (A fő autóműhely) ---
// Beimportáljuk a modult a relatív útvonal (./) megadásával
const matek = require('./matek');

console.log("5 + 3 =", matek.osszeadas(5, 3));
console.log("10 - 4 =", matek.kivonas(10, 4));

// Futtatás: node index.js
```

8. Függvények és visszatérési értékek szerveroldalon

Mi az a Callback (Visszahívó) függvény?

Korábban beszéltünk a Node.js aszinkron működéséről (pincér hasonlat). A kérdés az, honnan tudja a pincér, hogy a konyhában mikor készült el az étel, ha közben más asztaloknál van? A válasz a Callback! A programozásban egy függvénynek paraméterként nemcsak szöveget vagy számot tudunk átadni, hanem egy másik függvényt is. Képzeld el úgy, mint egy éttermi csipogót (pager). Leadod a rendelést, és kapsz egy csipogót (ez a callback függvény). Amikor elkészül az étel, a szakács 'meghívja' a csipogót. Így működik a fájlolvasás is: megmondjuk a Node.js-nek, hogy olvasson be egy 5 Gigabájtos fájlt, és adunk neki egy callback függvényt, amit akkor kell meghívnia, ha végzett, hogy addig is futhasson tovább a szerverünk.

Futtatható példa: functions.js

```
// --- functions.js ---
// 1. Alapértelmezett paraméterek
// Ha a híváskor nem adunk meg nevet, "Vendég" lesz az alapértelmezett, nem fog hibát
// dobni.
function koszontes(nev = "Vendég") {
    console.log(`Szia ${nev}! Üdv a szerveren.`);
}
koszontes("Gergő"); // Szia Gergő!
koszontes();        // Szia Vendég!

// 2. Callback (Visszahívó) függvény megvalósítása
function adatLekeres(callbackFuggveny) {
    console.log("\nKérjük az adatokat az adatbázistól... (ez beletelhet kis időbe)");

    // Szimuláljuk, hogy megjött az adat
    let letoltottAdat = "Titkos jelszó: 12345";

    // Amikor kész a "munka", meghívjuk a kapott függvényt az eredménnyel
    callbackFuggveny(letoltottAdat);
}

// Függvény hívása, egy anonim (nyíl)függvény átadásával
adatLekeres((eredmeny) => {
    console.log("A betöltés befejeződött! Kapott adat:", eredmeny);
});

// Futtatás: node functions.js
```

9. Referencia szerinti átadás (Memóriakezelés)

Érték (Primitív) vs Referencia

Ez a téma sok fejfájást okoz, ha nem értjük meg! A számítógép memóriáját képzeljük el úgy, mint a postafiókokat. Ha létrehozunk egy egyszerű számot (pl. `let a = 10`), a szám bekerül egy dobozba. Ha kiadjuk a `let b = a` parancsot, a számítógép lemásolja a 10-est, és beteszi egy új dobozba. Mindkettő független egymástól. Ezek a Primitív típusok. Azonban az objektumok (pl. egy teljes diák adatlapja) nagyok. A számítógép ezeket nem másolhatja. Ha kiadjuk a `let user2 = user1` parancsot, a gép NEM másolja le az adatot. Helyette a `user2` kap egy 'térképet' (referenciát), ami arra a postafiókra mutat, ahol az eredeti objektum van. Mivel mindkettő ugyanarra a dobozra mutat, ha az egyiket módosítod, a másik is megváltozik! Ezt a Spread (...) operátorral (sekély másolással) tudjuk kikerülni.

Futtatható példa: memory.js

```
// --- memory.js ---
// 1. Primitív típus (Érték szerinti másolás)
let a = 10;
let b = a; // Készül egy igazi másolat
b = 20;    // Csak 'b' változik meg
console.log("A értéke:", a); // 10 maradt!

// 2. Referencia típus (Memóriacím átadása - A VESZÉLY!)
const user1 = { nev: "Peti" };
const user2 = user1; // NEM készül másolat, csak ugyanoda mutatnak!

user2.nev = "Gergő"; // Megváltoztatjuk user2 nevét...
console.log("User1 neve:", user1.nev); // Hupsz! user1 is "Gergő" lett, mert egy
helyen vannak!

// 3. A helyes megoldás: Sekély másolás Spread (...) operátorral
const eredetiAuto = { marka: "Ford" };
// A három pont szétszedi (kicsomagolja) az eredetit és beleteszi egy új, független
objektumba
const ujAuto = { ...eredetiAuto };

ujAuto.marka = "Opel";
console.log("\nEredeti autó:", eredetiAuto.marka); // "Ford" maradt! Sértetlen.
console.log("Új autó:", ujAuto.marka);           // "Opel"

// Futtatás: node memory.js
```

10. Adatmodellezés objektumokkal

Biztonságos közlekedés az adatokban

A szerver gyakran kap be beágyazott objektumokat, például amikor lekérünk egy felhasználót, az tartalmazhatja a lakcímét is. A Destrukturálás (kicsomagolás) egy kényelmi funkció, amivel másodpercek alatt kivethetünk értékeket változókba anélkül, hogy hosszú útvonalakat gépelnénk (pl. `diakAdat.kapcsolat.email` helyett csak `'email'`). Az Opcionális láncolás (`?.`) pedig a defenzív (védekező) programozás eszköze. Ha megpróbálunk hivatkozni egy adatra, ami nincs is ott, a szerver csúnya hibával leáll (összeomlik). A `'?.'` használatával jelezzük a gépnek: 'Csak akkor menj tovább, ha ez létezik, különben állj meg, és ne omolj össze.'

Futtatható példa: `objects.js`

```
// --- objects.js ---
const diakAdat = {
  nev: "Béla",
  osztaly: "10.B",
  kapcsolat: { email: "bela@suli.hu" } // Figyelem! Nincs telefon megadva!
};

// 1. Objektum Destrukturálás (Kicsomagolás)
// Nem kell mindig kiírni, hogy diakAdat.nev, egyből kiszedjük változókba!
const { nev, osztaly } = diakAdat;
console.log(`A diák neve: ${nev}, osztálya: ${osztaly}`);

// 2. Opcionális Láncolás (Optional Chaining: '?.' )
// Szeretnénk lekérni Béla telefonszámát.

// Hagyományos, veszélyes mód:
// let telefon = diakAdat.kapcsolat.telefon.szam; // EZ LEFAGYASZTJA A SZERVERT, mert
// a 'telefon' nem létezik!

// Biztonságos, modern mód a ?. jellel:
// Ha valami hiányzik a láncban, egyszerűen megáll, és fagyás helyett 'undefined'-ot
// ad.
let telefonBiztonsagos = diakAdat.kapcsolat?.telefon?.szam;
console.log("Lekért telefonszám:", telefonBiztonsagos); // undefined (A szerver
// túlélte!)

// Futtatás: node objects.js
```

11. JSON formátum (JSON.stringify, JSON.parse)

A JSON: Az internet közvetítőnyelve

Amikor a backend (Node.js) és a frontend (pl. egy React weboldal, vagy egy mobil app) beszélgetni akar egymással, van egy probléma: lehet, hogy a mobil app Java-ban íródott, a backend pedig JS-ben. A memóriában lévő objektumokat nem lehet csak úgy átküldeni az interneten (hálózati kábelén). Szükség van egy közös formátumra, amit minden nyelv megért. Ez a JSON (JavaScript Object Notation). A JSON egyszerű, formázott sima szöveg (string). Szöveget pedig könnyedén tudunk küldeni hálózaton. A folyamatot, amikor az objektumból szöveget csinálunk, Serializációnak nevezzük. Amikor a beérkező szöveget visszaalakítjuk használható objektummá, az a Deszerializáció.

Futtatható példa: json_handling.js

```
// --- json_handling.js ---
const diakObjektum = { nev: "Kata", kor: 16 };

// 1. JS objektumból -> JSON Szöveg (Serializáció)
// Ezt csináljuk, amikor az adatot ki akarjuk küldeni az internetre.
const jsonSzoveg = JSON.stringify(diakObjektum);
console.log("Generált JSON szöveg hálózathoz:", jsonSzoveg);

// 2. JSON Szövegből -> JS Objektum (Deszerializáció) biztonságosan
// Mi van, ha a kliens rossz, hibás JSON szöveget küld?
const hibasJson = '{"nev": "László", "kor": 30, }'; // Hibás! (Vessző van a végén)

// A JSON.parse() kritikus pont, mert ha rossz a bejövő szöveg, leállítja a szerveret!
// Ezért MINDIG 'try-catch' blokkba (kivételkezelésbe) tesszük.
try {
  // A gép megpróbálja átalakítani...
  const bejovoAdat = JSON.parse(hibasJson);
  console.log("Sikeres beolvasás!", bejovoAdat.nev);
} catch (hiba) {
  // Ha a 'try' részben hiba történik (mint most), azonnal ide ugrik a kód!
  // Így a programunk biztonságosan tovább fut, tudunk válaszolni a hívónak.
  console.error("Hiba történt: Rossz a bejövő JSON formátum!");
}

// Futtatás: node json_handling.js
```

12. Objektum-tömbök mint memóriabeli adatszerkezet

Miért jobb a tömbfüggvények a for ciklusnál?

Ha van egy 100 elemű listánk (tömbünk) diákokkal, és ki akarjuk keresni az 5-ös tanulókat, egy hagyományos 'for' ciklussal sok sort kellene írni (létrehozni egy üres tömböt, indítani a ciklust, beletenni az if-et, és belenyomni az elemet a tömbbe). Ezt Imperatív programozásnak hívjuk, amikor megmondjuk a gépnek, **hogyan** csinálja meg lépésről lépésre. A modern JavaScript Deklaratív megközelítést használ beépített tömbfüggvényekkel (filter, map). Itt csak azt mondjuk meg, **mit** akarunk elérni (pl. filter = szűrjük le). Ez a háttérben elvégzi a ciklust helyettünk. Sokkal olvashatóbb, tisztább kódot eredményez, ami a profi szoftverfejlesztés alapkövetelménye.

Futtatható példa: arrays.js

```
// --- arrays.js ---
const termek = [
  { nev: "Kenyér", ar: 500, raktaron: true },
  { nev: "Tej", ar: 400, raktaron: true },
  { nev: "TV", ar: 150000, raktaron: false },
  { nev: "Csoki", ar: 800, raktaron: true }
];

// 1. Szűrés (filter)
// Végigmegy az összes elemen. Ha a belső nyílfüggvény IGAZ-at ad vissza,
// megtartja az elemet, ha HAMIS-at, kihagyja. Létrehoz egy ÚJ listát.
const olcsoTermek = termek.filter(termek => {
  return termek.ar < 600 && termek.raktaron === true;
});

console.log("Olcso és elérhető termékek:");
console.table(olcsoTermek);

// 2. Átalakítás (map)
// Végigmegy a listán, és amilyen formátumot visszaadunk (return),
// azzal az új formátummal tölti fel az új tömböt.
// Például itt most "kidobjuk" az árakat, és egy tömböt csinálunk amiben CSAK a nevek
// vannak:
const termekNevek = termek.map(termek => {
  return termek.nev;
});

console.log("\nCsak a termékek nevei listába szedve:");
console.log(termekNevek);

// Futtatás: node arrays.js
```